

Código Secreto

Campo	Detalle
Autor	Cesar Sebastian Zambrana Ventura
Institución	Universidad Católica Boliviana "San Pablo", sede Santa Cruz de la Sierra
Materia	Introducción a la Programación 2026

1. Objetivo

El juego consiste en descubrir el código secreto 58274.

El jugador tiene 10 intentos. Durante la partida puede probar códigos, pedir algunas ayudas o rendirse.

Cuando descubre el código, todavía debe completar un desafío final para ganar el juego completo.

2. Reglas

1. El código secreto tiene 5 dígitos.
2. Los dígitos del código no se repiten.
3. La partida tiene máximo 10 intentos.
4. Desde el inicio se muestra un dato gratis: la posición 3 vale 2.
5. El jugador puede usar 2 revelaciones exactas.
6. El jugador puede usar 3 pistas suaves.
7. Después de descubrir el código aparece un desafío final.
8. El puntaje baja cuando se usan intentos, ayudas o se falla en el desafío final.

3. Organización de codigo.cpp

El archivo está organizado de forma simple:

```
1. biblioteca iostream
2. prototipos de funciones
3. main
4. funciones de apoyo
5. función jugar
```

La idea es que primero se vea qué funciones existen y después se vea cómo trabaja cada una.

El main solo controla el menú principal:

```
while (opcion != 3 && cin.eof() == false) {
    mostrar menu
    leer opcion
    ejecutar la opcion elegida
}
```

Esto hace que el programa sea fácil de seguir: mientras el jugador no elija salir, el menú se sigue mostrando.

4. Lectura de datos

La función leerEntero sirve para leer números de forma más segura.

Si el jugador escribe un número, el programa sigue normal.

Si escribe texto, por ejemplo hola, el programa muestra un mensaje y no se queda trabado.

```
if (leerEntero(opcion) == false) {
    continue;
}
```

Esto evita errores comunes cuando se usa cin.

5. Procesar dígitos

El juego trabaja con números de 5 dígitos. Para revisar esos dígitos se usan divisiones y módulos.

Para obtener el último dígito:

```
digito = numero % 10;
```

Para quitar el último dígito:

```
numero = numero / 10;
```

Ejemplo:

Paso	Número antes	Dígito extraído	Número después
1	58274	4	5827
2	5827	7	582
3	582	2	58
4	58	8	5
5	5	5	0

También existe la función `obtenerDigitoEnPosicion`, que permite pedir un dígito por su posición.

Por ejemplo, en 58274, la posición 3 es 2.

6. Funciones principales

Función	Qué hace
<code>mostrarReglas</code>	Muestra las reglas del juego.
<code>jugar</code>	Controla toda la partida.
<code>leerEntero</code>	Lee un número y evita errores con texto.
<code>obtenerDigitoEnPosicion</code>	Devuelve un dígito según su posición.
<code>existeDigito</code>	Revisa si un dígito está dentro de un número.
<code>tieneCincoDigitos</code>	Revisa que el intento tenga 5 dígitos.
<code>tieneDigitosDiferentes</code>	Revisa que no haya dígitos repetidos.
<code>intentoEsValido</code>	Une las validaciones del intento.
<code>contarLugaresExactos</code>	Cuenta dígitos correctos en el mismo lugar.
<code>contarCorrectosEnOtroLugar</code>	Cuenta dígitos correctos pero en otra posición.
<code>sumarDigitos</code>	Calcula la pista de la suma.
<code>contarPares</code>	Calcula cuántos dígitos pares hay.
<code>calcularClaveFinal</code>	Calcula la clave del desafío final.
<code>calcularPuntaje</code>	Calcula el puntaje final.

7. Validación del intento

Un intento solo es válido si cumple dos condiciones:

1. Tiene 5 dígitos.
2. No repite dígitos.

Por eso existe esta función:

```
bool intentoEsValido(int intento)
```

La validación se separó para que `jugar` no tenga demasiadas instrucciones juntas.

Así es más fácil explicar el programa:

```
primero leo el intento
después reviso si es válido
si es válido, cuenta como intento
si no es válido, no cuenta como intento
```

8. Respuesta después de un intento

Cuando el jugador prueba un código, el programa compara ese intento con el código secreto.

El programa muestra dos datos:

```
lugares exactos
dígitos correctos en otro lugar
```

Ejemplo:

Código secreto	Intento	Resultado
58274	12345	lugares exactos: 0 dígitos correctos en otro lugar: 3

Esto pasa porque los dígitos 2, 4 y 5 sí están en el código, pero están en otras posiciones.

9. Ayudas

El juego tiene dos tipos de ayuda.

Las revelaciones muestran un dato exacto:

```
posición 1 = 5
posición 5 = 4
```

Las pistas suaves dan información general:

```
suma de dígitos = 26
hay 3 pares y 2 impares
el código es mayor que 50000
```

Estas ayudas no gastan intentos, pero sí bajan el puntaje si el jugador gana.

10. Desafío final

Después de descubrir `58274`, el jugador debe formar una clave final.

La regla es:

```
mayor par, mayor impar, siguiente par, siguiente impar
```

Con 58274:

```
Pares: 8, 4, 2
Impares: 7, 5
Clave final: 87452
```

La función `calcularClaveFinal` hace esto buscando dígitos desde los mayores hacia los menores.

No usa arreglos. Solo usa números, condiciones y bucles.

11. Puntaje

El puntaje empieza en 100.

Si el jugador no gana el juego completo, el puntaje final es 0.

Si gana, se descuentan puntos por:

```
intentos usados
revelaciones usadas
pistas usadas
errores en el desafío final
```

En el código, esos descuentos se aplican así:

Concepto	Descuento
Intento usado	4 puntos
Revelación usada	12 puntos
Pista usada	8 puntos
Error en el desafío final	10 puntos

12. Compilar y ejecutar

Para compilar:

```
g++ -std=c++17 -Wall -Wextra -pedantic codigo.cpp -o build/codigo_secreto
```

Para ejecutar:

```
./build/codigo_secreto
```